Indice

1 MultiWay Search Trees & B-Trees **2**
1.1 Multiway Search Tree 2
1.1.1 Useful Property 2
1.2 a-bTree 2
1.2.1 Altezza di un abTree 3
1.2.2 Ricerca in un abTree 4
1.2.3 Inserimento in un abTree 5
1.2.4 Cancellazione abTree 5
1.3 B-tree 5
1.4 Differenze tra MWST, ab-Tree, Btree 5
1.4.1 MWST 5
1.4.2 a-bTree 6
1.4.3 Btree 6
1.5 Quindi... 7

Maps

Appunti di Programmazione

28 marzo 2026

1 MultiWay Search Trees & B-Trees

1.1 Multiway Search Tree

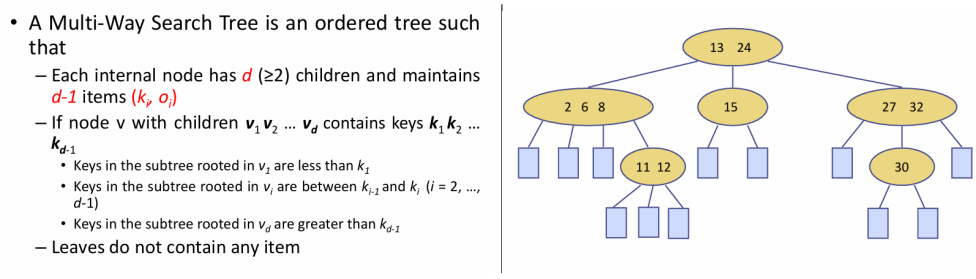

Figura 1: MultiWay Tree

1.1.1 Useful Property

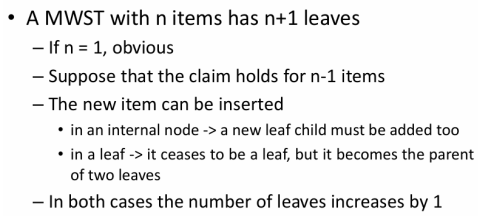

Figura 2: Useful Property

1.2 a-bTree

A differenza degli alberi binari, dove ogni nodo ha al massimo 2 figli, qui ogni nodo può averne molti di più, rendendo l'albero molto più "basso" e largo.

Perché se b è pari, allora:

$$2 \leq a \leq b/2$$

mentre se b è dispari, allora:

$$2 \leq a \leq (b-1)/2$$

???

Serve a garantire che, dopo una divisione, entrambi i nuovi nodi abbiano almeno a figli.

- **Se b è pari (es. $b = 6$)** :Dividendo $b+1$ figli (cioè 7), ottieni $\lfloor 7/2 \rfloor = 3$ e $\lceil 7/2 \rceil = 4$. Per far sì che entrambi siano validi, il minimo a deve essere al massimo 3. $(b/2)$.

- **Se b è dispari (es. $b = 5$)**: Dividendo $b + 1$ figli (cioè 6), ottieni $6/2 = 3$ e 3. Il minimo a deve essere al massimo 3. $((b + 1)/2$, che è coerente con la formula $(b-1)/2$ se consideri il numero di chiavi invece dei figli).

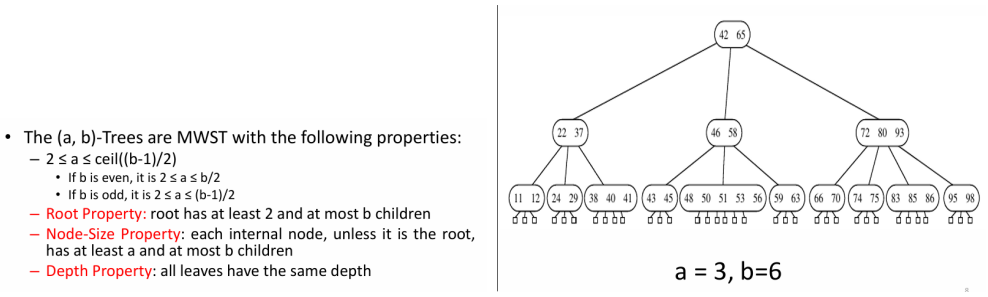

Figura 3: a-b Tree

1.2.1 Altezza di un abTree

Considerando n elementi memorizzati e sapendo che un albero di ricerca multi-way presenta sempre $n + 1$ foglie, la struttura è vincolata dai seguenti parametri:

- **Configurazione Minima (Worst Case)**: si verifica quando ogni nodo interno ha il numero minimo di figli (a). La radice ha 2 figli e i livelli successivi si ramificano con fattore a, producendo un numero di foglie pari a $2a^{h-1}$.

- **Configurazione Massima (Best Case)**: Si verifica quando ogni nodo è saturo con b figli. In questo scenario, il numero di foglie raggiunge il valore b^h.

Questi vincoli si traducono nella disuguaglianza fondamentale $2a^{h-1} \leq n+1 \leq b^h$. Attraverso il calcolo logaritmico, si isola la variabile dell'altezza ottenendo la relazione:

$$(h-1)\log a + \log 2 \leq \log(n+1) \leq h \log b$$

Da questa derivazione si conclude che l'altezza h è:

- Limitata superiormente da $O(\log n / \log a)$.
- Limitata inferiormente da $\Omega(\log n / \log b)$.

In sintesi, l'uso di basi logaritmiche elevate (a e b grandi) permette di mantenere l'altezza estremamente ridotta, ottimizzando le prestazioni su grandi dataset.

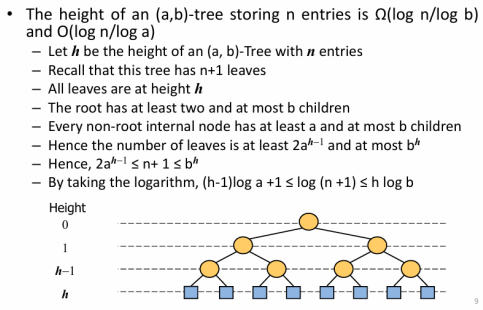

Figura 4: Altezza di un abTree

1.2.2 Ricerca in un abTree

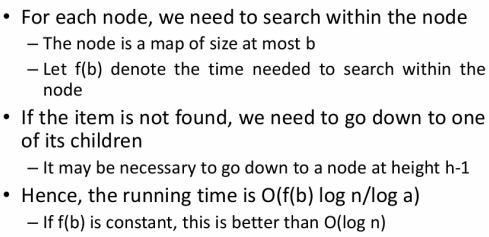

Figura 5: Ricerca in un abTree

1.2.3 Inserimento in un abTree

1. To insert an item (***k***, ***o***), search for the node ***v*** that should contain ***k***
2. If ***v*** contains less than b-1 items, add the new item to it
3. Otherwise, an **overflow** occurs
   - Consider the keys saved in node ***v***, plus key ***k***
   - We **split** them in three parts: the median key (***k'***), keys smaller than the median and keys larger than the median
   - With keys smaller than the median we create a new node
     - It is a valid node because it contains at least ceil((b-1)/2)-1 ≥ a-1
   - The same holds for keys larger than the median
   - The median key is inserted in the parent ***p*** of ***v*** with a pointer to the newly created nodes
     - Then, we repeat from step 2 with ***p*** in place of ***v***, and (***k'***, ***o'***) as the new item to insert
     - If ***v*** is the root, then a new root is created

Figura 6: Inserimento in un abTree

1.2.4 Cancellazione abTree

1. To delete an item (***k***, ***o***), search for the node ***v*** that contains ***k***
2. If either the child on the left or the right of ***k*** is non-empty or ***v*** is the root, then
   - Let ***k'*** be the smallest key in the subtree to the right of ***k*** (or largest at the left of ***k***)
   - Replace the item with key ***k*** with the item whose key is ***k'***
   - Delete item with key ***k'*** from its subtree
3. If children are empty, and v has >a-1 items, then delete, o.w. an **underflow** occurs
   - If the node ***w*** on the left of ***v*** has more than a-1 items, then perform a **transfer**
     - Let ***k'*** be the key saved in the parent ***p*** that is in between the keys contained in ***w*** and ***v***
     - Let ***k''*** be the largest key saved in ***w***
     - Delete ***k*** from ***v***, delete ***k''*** from ***w***, replace ***k'*** with ***k''*** in ***p***, and add ***k'*** in ***v***
   - If the node ***w*** on the right of ***v*** has more than a-1 items, then perform a **transfer**
     - The opposite of above operation
   - Otherwise perform a **fusion**
     - Let ***w*** the node on the left of ***v*** and ***p*** its parent
     - Let ***k'*** be the key saved in ***p*** in between the keys of ***w*** and ***v***
     - Create a new node containing all keys of ***v*** except ***k***, ***w*** and key ***k'***
       - It is a valid node because it contains (a-2)+a-1+1 = 2a-1 ≤ b-1 items
     - Remove ***k'*** from ***p***
       - If ***p*** is the root and ***k'*** is its unique element, then the new node becomes the new root

16

Figura 7: Cancellazione in un abTree

1.3 B-tree

A B-tree of order d is an (a,b)-tree with a = ceil((b-1)/2) and b = d

- Search has complexity O(f(d)log n/(log d – 1))
- Insert/Delete have complexity O(g(d)log n/(log d – 1))

Figura 8: B-tree

1.4 Differenze tra MWST, ab-Tree, Btree

1.4.1 MWST

Il Multiway Search Tree (il concetto base) è la definizione più libera. Un nodo può avere un numero di figli k qualsiasi. Se un nodo ha k figli, contiene $k - 1$ chiavi.

- **Limiti**: Nessuno. Non ci sono parametri a o b fissi.
- **Problema**: L'albero può diventare sbilanciato (molto alto e stretto), vanificando la velocità di ricerca.

1.4.2 a-bTree

Quando crei l'albero, fissi due numeri interi, a e b. Una volta scelti, questi parametri non cambiano più per tutta la vita dell'albero.

- **I Vincoli sui parametri**: per poter funzionare (e permettere lo split), devi scegliere a e b tali che:

  $$2 \leq a \leq \left\lceil \frac{b-1}{2} \right\rceil$$

  Hint: $\lceil 3.4 \rceil = 4$.
  Ad esempio, se fissi $b = 10$, puoi scegliere un a tra 2 e 5. Se scegli $a = 2$, rimarrà 2 per sempre.

- **I Vincoli sui nodi**: Ogni nodo interno deve avere un numero di figli f compreso tra i parametri fissati: $a \leq f \leq b$.
- **Flessibilità**: È un modello teorico. Ti permette di decidere quanta "libertà" dare ai nodi (se a è piccolo, i nodi possono essere quasi vuoti; se a è vicino a $b/2$, i nodi devono essere più pieni).

1.4.3 Btree

Il B-Tree è un (a, b)-tree dove non hai libertà di scegliere a. Scegli solo il grado (o ordine) d, che corrisponde a b.

- **I Parametri fissi**
  - $b = d$
  - $a = \lceil d - 1/2 \rceil$
- **La Filosofia**: Il B-Tree è "avaro" di spazio. Obbliga ogni nodo a essere pieno almeno al 50%. Se un nodo scende sotto questa soglia (underflow), l'albero si auto-ripara immediatamente tramite fusioni o prestiti.

1.5 Quindi...

Il MWST è la versione generale. Un nodo con k chiavi ha $k+1$ figli.

L'a-bTree definisce dei limiti sul numero di figli di ogni nodo:

- **Numero massimo di figli**: b.
- **Numero minimo di figli**: $2 \leq a \leq \lceil \frac{b-1}{2} \rceil$.

Il Btree è un a-bTree che impone il numero minimo al valore massimo consentito:

- **Numero massimo di figli**: b.
- **Numero minimo di figli**: $a = \lceil \frac{b-1}{2} \rceil$.

Sotto il profilo dell'inserimento, le due strutture sono meccanicamente identiche: entrambe gestiscono l'eccesso di chiavi tramite lo *split*.

La vera differenza emerge nella *Cancellazione*
È qui che i due alberi si comportano diversamente:

- Se in un (a, b)-tree con $a = 2$ cancelli chiavi finché in un nodo ne resta solo una (2 figli), l'albero non fa nulla. Il nodo resta lì, mezzo vuoto.
- In un B-tree con $a = 50$, appena scendi sotto le 49 chiavi (50 figli), l'albero "va nel panico". Deve attivare immediatamente operazioni di Merge (fusione) o Transfer (prestito da un vicino) per riportare il nodo sopra il 50%.